Advanced Python: Iterators, Generators and Recursion

1. Recursive Algorithms and Call Stacks

Recursion is a technique where a function calls itself to solve smaller subproblems until a base condition is reached.

```
def recursive_fibonacci(n: int) -> int:
    """Calculates the n-th Fibonacci number with recursive branching."""
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    # Recursive reduction
    return recursive_fibonacci(n - 1) + recursive_fibonacci(n - 2)

# Sample Execution
if __name__ == '__main__':
    values = [recursive_fibonacci(i) for i in range(8)]
    print(f'Fibonacci Sequence: {values}')
```

2. Algorithmic Complexity Analysis

The naive recursive Fibonacci algorithm exhibits exponential time complexity O(2^n). Using memoization or dynamic programming reduces execution to linear time O(n).